Chapter 5: System Design

5.1 Introduction

The analysis work in Chapter 4 established what the system must do — the actors, the data flows, the use cases, and the requirements that bound the design. This chapter records the decisions about *how* those requirements will be satisfied: what layered structure the application follows, how the database is organised to support both operational and longitudinal use, what the machine learning pipeline looks like from raw satellite observation to health classification, and what each page of the interface presents to a farmer. Getting the architecture right at this point matters because, as **?** notes, design errors are the most expensive to fix later, since every implementation phase builds directly on top of them.

The overarching design philosophy is modular separation. Each layer has one job, exposes a clean interface to the layers around it, and does not reach into the internals of anything else. That principle runs through every decision in this chapter — from the way GEE and Open-Meteo calls are isolated in their own modules, to the way Python model training is entirely separate from the JavaScript inference that runs at deployment time. Keeping responsibilities narrow is what makes a system like this maintainable by a single developer and extensible in the future without dismantling what already works.

5.2 System Design

Overall Design Approach

The system follows a four-layer modular architecture. Satellite data retrieval, backend coordination, intelligence processing, and browser-based presentation each occupy their own layer. Communication between layers flows through defined interfaces rather than direct internal access.

- **Satellite and weather data acquisition layer**
  - This layer's only job is to accept a GeoJSON polygon and a date range and return a structured feature set. It holds no business logic and nothing about users or fields.
  - Google Earth Engine handles the satellite side. The `COPERNICUS/S2_SR_HARMONIZED` collection is queried with the field polygon as the region of interest, filtered to scenes with cloud cover below 20% using the QA60 band, and sampled over

the analysis window. Per-pixel NDVI is computed at 10-metre resolution from Band 8 (NIR) and Band 4 (Red): NDVI $= (B_8 - B_4)/(B_8 + B_4)$. The spatial pixel array is retained for heatmap rendering. (**??**)

- Weather data is retrieved in parallel from the Open-Meteo API using the field centroid coordinates. Daily temperature and precipitation records over the same window are aggregated into average temperature and total rainfall. Running these two API calls concurrently reduces overall analysis latency. (**?**)
- The layer returns five engineered values plus a pixel-level NDVI grid. Nothing that calls this layer needs to know anything about HTTP requests, GEE authentication, or how NDVI is computed.

- **Backend processing and storage layer**
  - Next.js API routes receive incoming requests, validate the session and ownership, orchestrate calls to the acquisition and intelligence layers, and persist results to the database. Validation is the first thing that runs on every request — an unauthenticated or unauthorised request never reaches any computation.
  - Storing raw feature values alongside derived outputs (health status, risk flags) is a deliberate design choice. It means a stored analysis record can be re-evaluated if the model is retrained, and the reasoning behind any past recommendation can be traced back to the data that produced it. (**?**)
  - The route handler itself is deliberately thin. The satellite module, inference module, and recommendation module each live in separate files, so any one of them can be replaced or updated without touching the request-handling logic.

- **Intelligence layer**
  - The logistic regression classifier runs entirely in JavaScript using the exported JSON weight file. No Python interpreter is needed at runtime — the model coefficients and intercepts are loaded once at server start and reused across all inference requests. (**?**)
  - Classification returns one of `HEALTHY`, `MODERATE_STRESS`, or `HIGH_STRESS`. The NDVI trend slope is separately classified as `IMPROVING`, `STABLE`, or `DECLINING` from a threshold applied to the raw slope value.
  - The recommendation engine evaluates four independent rules against the feature values and health status. Rules are not mutually exclusive: a field with low NDVI, low rainfall, high variance, and elevated temperature fires all four applicable rules simultaneously. (**?**)

- **Presentation layer**

- The browser application handles all farmer-facing interaction through server-rendered Next.js 14 pages with TypeScript. Registration, login, field creation, analysis triggering, and result review are each their own page, making the interface navigable even on slow mobile connections.
- The interactive map uses React-Leaflet with an ESRI World Imagery satellite tile layer as the basemap, so farmers can draw polygon boundaries directly over recognisable aerial imagery of their own land. The Leaflet-Draw plugin provides the drawing tools, restricted to the polygon type only.
- Health status is communicated primarily through colour — green for Healthy, amber for Moderate Stress, red for High Stress. **?** identifies plain, immediately readable outputs as one of the strongest predictors of whether farmers actually use a decision support tool. That finding shaped the entire results page design.

5.2.1 How will the System Work?

The main analysis workflow runs from the farmer selecting a field through to a complete results page being displayed. The steps below trace that flow in the order it actually executes.

- **Authentication and session management**
  - The farmer logs in with email and password. The backend validates the submitted password against the stored bcrypt hash and, on success, issues an HTTP-only session cookie. Every API request thereafter is checked against that cookie before any processing begins.
- **Field creation with interactive boundary drawing**
  - On the Create Field page, a satellite map loads centred on Zimbabwe. The farmer enters a field name and activates the polygon drawing tool. Clicking around the field perimeter on the satellite imagery closes the polygon; it can be edited or redrawn before saving.
  - On submission, the GeoJSON polygon is sent to `/api/field`. The backend computes the field area using the Shoelace formula, reverse-geocodes the centroid to a place name through the OpenStreetMap Nominatim API, and stores the record.
- **Satellite data retrieval**
  - When the farmer triggers an analysis, the stored polygon is retrieved and passed to the satellite module. The GEE service account authenticates and queries

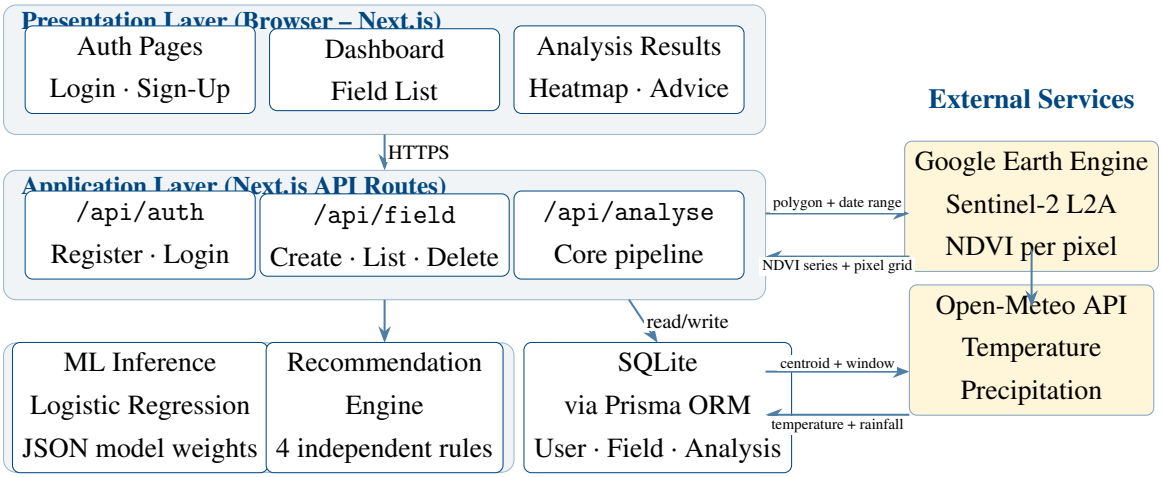
`COPERNICUS/S2_SR_HARMONIZED` over a 30-day lookback window, returning the per-pixel NDVI series and spatial grid. The Open-Meteo call runs in parallel.

- **Feature engineering, classification, and recommendation**
  - Mean NDVI, trend slope, within-field variance, average temperature, and total rainfall are assembled into the feature vector. The logistic regression classifier scores each class and returns the highest-scoring label. The recommendation engine then evaluates all four rules and returns the matched advice strings.
- **Storage and display**
  - The complete analysis record is written to the database. The formatted response — health status, feature values, risk flags, recommendations, and heatmap pixel array — is returned to the browser. The results page renders the health badge, recommendation cards, environment summary, NDVI heatmap overlay, and the NDVI history chart.

5.3 Solution Architecture

Figure **??** presents the proposed system architecture. The four internal layers stack vertically, with the two external data services — Google Earth Engine and Open-Meteo — connecting into the application layer from the right. All farmer interaction flows through the top presentation layer, and all persistence flows down to the data layer.

Figure 5.1: Proposed System Architecture — TobaccoGuard



The analysis route in the application layer is the most complex component in the architecture. It is the only point where both external API calls are made, it is the only caller of the ML inference and recommendation functions, and it is responsible for persisting the

complete result before returning the response to the browser. Everything else in the system is kept simple by design.

5.4 Database Modelling

Purpose of the Database

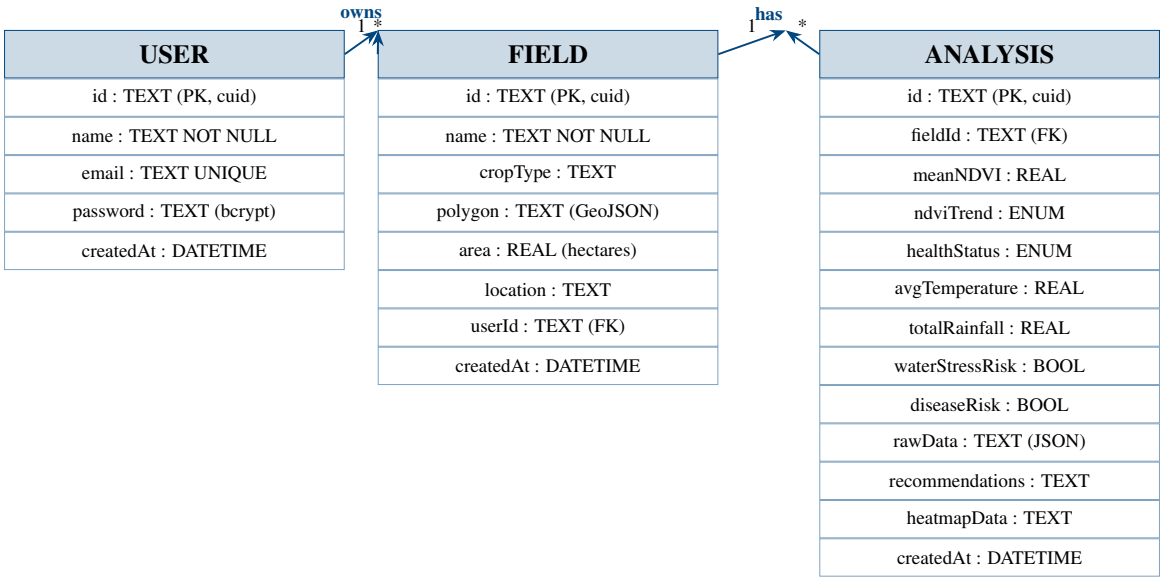
The database serves two roles. First, it is the operational backbone of the application — it stores user credentials, field polygons, and analysis results in a way that persists across sessions and across seasons. Second, it is the foundation for longitudinal monitoring: every analysis run for a field is stored as a distinct record with a timestamp, which means the system can surface historical NDVI trend data alongside any current reading. That time-series view is, in many ways, more useful to a farmer than any single analysis in isolation. (**?**)

- **Choice of database technology**
  - SQLite was selected over a server-managed database for the prototype. Its file-based architecture removes the deployment overhead of a separate database process, and Prisma's ORM layer abstracts the SQL entirely, making a future migration to PostgreSQL a single provider setting change. For a bounded prototype user base without high concurrent write volume, SQLite is sufficient. (**?**)
- **Core datasets**
  - The most critical data the system stores is the GeoJSON polygon for each field — it is the input to every analysis and cannot be reconstructed from any other record. Analysis records are the second most important: they store both the raw feature values and the derived outputs, so results are auditable and the model's reasoning can be traced to specific observations.
  - The three-tier hierarchy — User owns Fields, Fields accumulate Analyses — is the central relational structure. Cascade deletes flow downward: removing a field removes all its historical analyses; removing a user removes all their fields and analyses.

5.4.1 E-R Diagram

Figure **??** shows the entity-relationship diagram for the database. The three entities correspond directly to the three Prisma models in the schema.

Figure 5.2: Entity-Relationship Diagram — TobaccoGuard Database



5.4.2 Data Dictionary

The following tables define the fields, data types, and constraints for each entity in the database.

USER (Farmer account records)

Field	Data Type	Description	Constraints
id	TEXT	Unique identifier for the user account, auto-generated using `cuid()`	PK, unique, not null
name	TEXT	Farmer's full name as entered during registration	not null
email	TEXT	Farmer's email address, used for login	unique, not null
password	TEXT	bcrypt hash of the user's password, work factor 12	not null
createdAt	DATETIME	Timestamp of account creation, set automatically	not null

FIELD (Tobacco field records)

Field	Data Type	Description	Constraints
id	TEXT	Unique identifier for the field record	PK, unique, not null
name	TEXT	Field name as entered by the farmer	not null
cropType	TEXT	Crop grown in the field; defaults to `Tobacco`	default: Tobacco
polygon	TEXT	GeoJSON representation of the drawn field boundary	not null
area	REAL	Calculated field area in hectares using Shoelace formula	nullable
location	TEXT	Reverse-geocoded place name from field centroid	nullable
userId	TEXT	Reference to the owning user's account	FK → USER.id
createdAt	DATETIME	Timestamp of field record creation	not null

ANALYSIS (Crop health analysis records)

Field	Data Type	Description	Constraints
id	TEXT	Unique identifier for the analysis run	PK, unique, not null
fieldId	TEXT	Reference to the analysed field	FK → FIELD.id
meanNDVI	REAL	Mean NDVI across all cloud-free pixels in the analysis window	not null
ndviTrend	ENUM	Temporal NDVI trajectory: IMPROVING, STABLE, or DECLINING	not null
healthStatus	ENUM	ML classifier output: HEALTHY, MODERATE_STRESS, or HIGH_STRESS	not null
avgTemperature	REAL	Average daily temperature (°C) over the analysis window	not null

Field	Data Type	Description	Constraints
totalRainfall	REAL	Cumulative precipitation (mm) over the analysis window	not null
waterStressRisk	BOOL	True if rainfall < 10 mm AND mean NDVI < 0.45	not null
diseaseRisk	BOOL	True if NDVI spatial variance $>$ 0.05	not null
rawData	TEXT	JSON string storing raw feature values and data source metadata	not null
recommendations	TEXT	JSON array of plain-language recommendation strings	not null
heatmapData	TEXT	JSON array of `[lat, lng, ndvi]` triples for heatmap rendering	nullable
createdAt	DATETIME	Timestamp of analysis run	not null

5.4.3 Database Schema

The Prisma schema below defines all three models, the two enumerations, and the relationship constraints. The complete schema file is reproduced in full in Appendix **??**.

Listing 5.1: Prisma schema – core models (abridged)

```
model User {
  id         String   @id @default(cuid())
  name       String
  email      String   @unique
  password   String
  fields     Field[]
  createdAt  DateTime @default(now())
}

model Field {
  id         String     @id @default(cuid())
  name       String
  cropType   String     @default("Tobacco")
  polygon    String     // GeoJSON stored as JSON string
  area       Float?
  location   String?
  userId     String
```

```
  user       User        @relation(fields: [userId], references: [
     id])
  analyses   Analysis[]
  createdAt  DateTime    @default(now())
}

model Analysis {
  id               String        @id @default(cuid())
  fieldId          String
  field            Field         @relation(fields: [fieldId],
     references: [id])
  meanNDVI         Float
  ndviTrend        NDVITrend
  healthStatus     HealthStatus
  avgTemperature   Float
  totalRainfall    Float
  waterStressRisk  Boolean
  diseaseRisk      Boolean
  rawData          String
  recommendations  String
  heatmapData      String?
  createdAt        DateTime      @default(now())
}

enum NDVITrend    { IMPROVING  STABLE  DECLINING }
enum HealthStatus { HEALTHY  MODERATE_STRESS  HIGH_STRESS }
```

5.5 Algorithm Design

The analysis algorithm converts a GeoJSON polygon into a health classification and a set of farming recommendations through a six-step pipeline: ingest and validate → satellite and weather retrieval → feature engineering → ML inference → post-processing and risk flagging → storage and response. Each step is described below. (**?**)

5.5.1 Inputs to the Algorithm

- The algorithm receives a field record containing:
  - The GeoJSON polygon geometry (a `FeatureCollection` wrapping a single `Polygon` feature).

- The `fieldId` (used to verify ownership and store results).
- The authenticated user's session identifier.

- Derived from the polygon during the analysis run:
  - Sentinel-2 Level-2A per-pixel NDVI values over a 30-day window, drawn from the `COPERNICUS/S2_SR_HARMONIZED` GEE collection.
  - Daily temperature and precipitation for the same window from Open-Meteo.

5.5.2 Pre-processing and Validation

- **Request validation**
  - The session cookie is checked. If absent or invalid, the request is rejected with HTTP 401 before any computation begins.
  - The field record is fetched from the database filtering on both `fieldId` and `userId`. A record that doesn't belong to the authenticated user returns HTTP 404 rather than HTTP 403, to avoid confirming the existence of other users' fields.

- **Polygon parsing**
  - The stored polygon JSON string is parsed. If parsing fails, the request is rejected. The polygon coordinates are extracted and validated for minimum vertex count (at least 4, including the closing coordinate) before being submitted to GEE.

- **Satellite data quality**
  - GEE scenes are filtered to cloud cover below 20%. If fewer than two qualifying scenes exist in the 30-day window, the lookback is extended to 60 days. If still insufficient, the analysis proceeds with a data quality flag in the raw data JSON.

5.5.3 Feature Engineering

The five features used by the logistic regression model are engineered from the raw satellite and weather observations as follows.

- **Mean NDVI** (`mean_ndvi`): The arithmetic mean of all per-pixel NDVI values across all qualifying scenes in the analysis window. Pixels with NDVI below -0.1 (water, shadow) are excluded from the mean computation.

- **NDVI trend slope** (`ndvi_trend`): A linear regression slope fitted to the sequence of per-scene mean NDVI values ordered by acquisition date. A positive slope indicates improving vegetation; a negative slope indicates decline. The slope is classified as `IMPROVING` if > 0, `DECLINING` if < -0.01, and `STABLE` otherwise. (**?**)

- **NDVI variance** (`ndvi_variance`): The spatial variance of per-pixel NDVI values within the field polygon, computed from the pixel distribution in the most recent cloud-free scene. High variance indicates irregular within-field vegetation patterns, which may indicate patchy disease, pest damage, or waterlogging in isolated zones. (**?**)

- **Average temperature** (`avg_temperature_c`): Mean of daily maximum temperatures over the analysis window from Open-Meteo.

- **Total rainfall** (`total_rainfall_mm`): Sum of daily precipitation over the same window.

5.5.4 Model Selection and Responsibilities

A multinomial logistic regression classifier is used for the three-class crop health problem. The choice was informed by **?**, who reviewed forty machine learning studies in agriculture and found that logistic regression frequently matches more complex models when features are well-engineered and class boundaries are reasonably stable. The key advantages here are interpretability — the coefficient magnitudes show which features most influence each class — and deterministic inference, since there is no randomness in the forward pass at prediction time.

- **Responsibility 1 – Health classification**: Assigns the field to `HEALTHY`, `MODERATE_STRESS`, or `HIGH_STRESS` from the five-feature vector.

- **Responsibility 2 – Risk flagging**: Two binary flags are derived separately from the raw features using threshold rules. `waterStressRisk` fires when `totalRainfall` < 10 mm and `meanNDVI` < 0.45. `diseaseRisk` fires when `ndviVariance` > 0.05.

- **Responsibility 3 – Trend tracking**: The NDVI trend direction is stored alongside the health status as a secondary indicator for longitudinal review.

5.5.5 Inference and Post-processing Logic

1. **Score computation**: For each class j, a linear score is computed: $z_j = \mathbf{w}_j \cdot \mathbf{x} + b_j$, where $\mathbf{w}_j$ is the row of coefficients for class j, $\mathbf{x}$ is the feature vector, and b_j is the class intercept.

2. **Class selection**: The class with the highest score is selected as the prediction: $\hat{y} = \arg\max_j(z_j)$.

3. **Recommendation generation**: The four recommendation rules are evaluated independently:

   - **R1 – Water stress**: NDVI < 0.45 and rainfall $< 10\,\text{mm} \rightarrow$ irrigation recommended.
   - **R2 – Declining moderate**: trend = DECLINING and $0.45 \leq \text{NDVI} \leq 0.60 \rightarrow$ monitor closely.
   - **R3 – Stable healthy**: NDVI > 0.60 and trend = STABLE or IMPROVING $\rightarrow$ no action required.
   - **R4 – Disease signal**: variance > 0.05 and temperature $> 25\,^{\circ}\text{C} \rightarrow$ inspect for disease or pests.

4. **Heatmap generation**: The retained per-pixel NDVI grid is converted to a list of `[lat, lng, intensity]` triples, where intensity $= 1 -$ normalised NDVI (so that red corresponds to low NDVI / high stress, and green to high NDVI / healthy). Pixels are sampled at fixed intervals to keep the payload manageable.

5.5.6 Algorithm Pseudocode

```
Receive analysis request with fieldId and session cookie
Validate session cookie and ownership of fieldId
Retrieve field polygon from database
Parse GeoJSON polygon and extract coordinate array
Submit polygon to Google Earth Engine
  Filter COPERNICUS/S2_SR_HARMONIZED by polygon and 30-day window
  Apply QA60 cloud mask (< 20% cloud cover)
  Compute per-pixel NDVI = (B8 - B4) / (B8 + B4)
  Return NDVI time-series and spatial pixel grid
In parallel, request weather data from Open-Meteo
  For field centroid coordinates over same 30-day window
  Return daily temperature and precipitation
Compute engineered features:
  mean_ndvi       = mean of all qualifying pixels
  ndvi_trend      = linear regression slope over time-series
  ndvi_variance   = spatial variance of pixel distribution
  avg_temperature = mean of daily maximum temperatures
  total_rainfall  = sum of daily precipitation
```

```
Load JSON model weights (coefficients + intercepts)
For each class: compute z = dot(weights[j], features) + bias[j]
Select class with highest z as health status
Evaluate four recommendation rules against features and status
Generate heatmap point array from pixel grid
Write complete analysis record to database
Return response to browser (status, features, recommendations, heatmap)
```

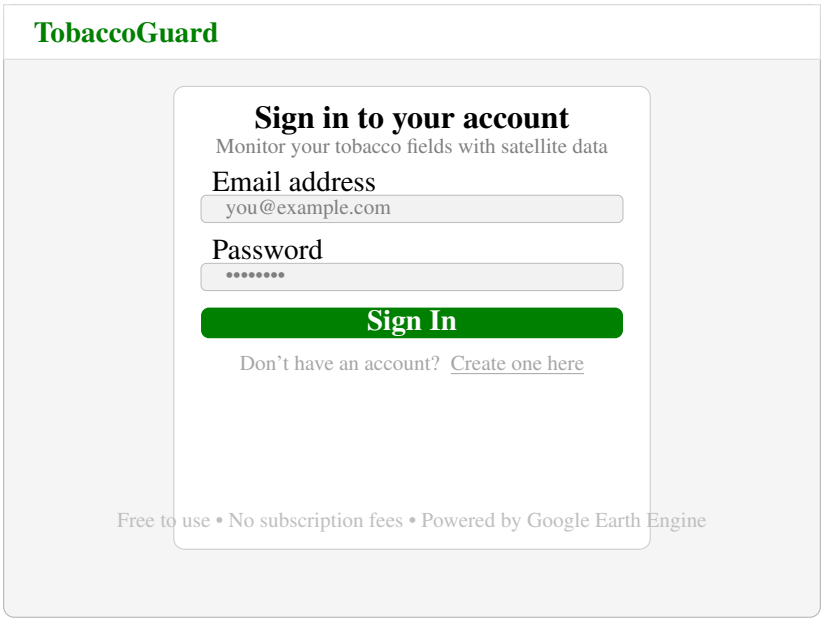
5.6 Interface Design

The interface was designed around one principle: the primary output of any page should be readable in under three seconds, without scrolling, by a farmer with basic digital literacy. That constraint pushed towards large, colour-coded status indicators, minimal text on the primary views, and details moved into expandable sections for users who want them. **?**

5.6.1 Login and Registration Interface

The Login and Sign-Up pages are deliberately simple. The farmer's task here is purely credential entry — the design should not distract from that with navigation or secondary content.

Figure 5.3: Wireframe — Login Page

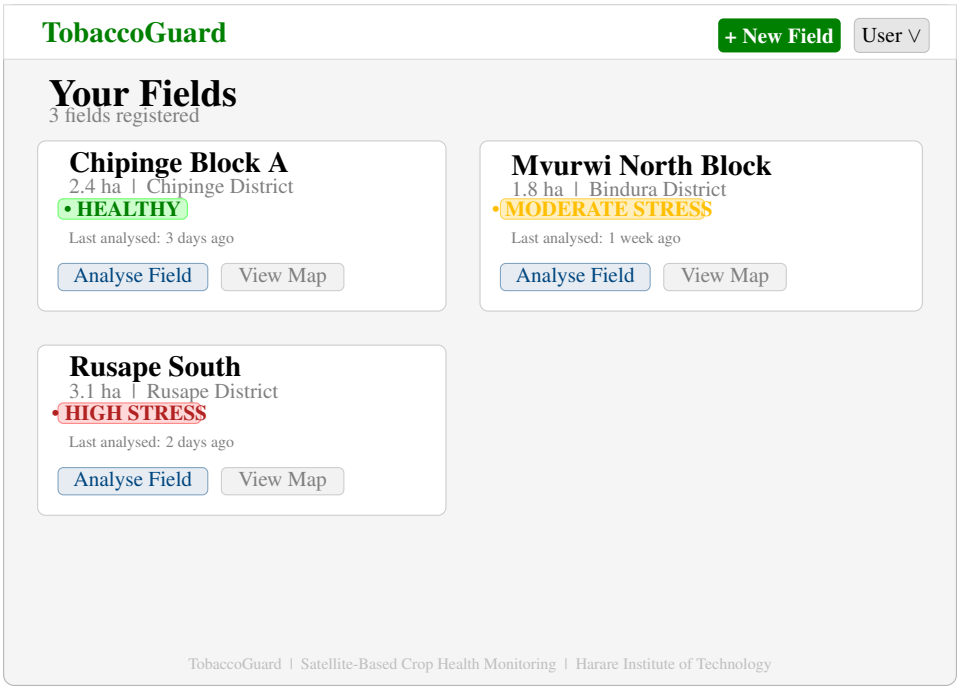


5.6.2 Dashboard Interface

The Dashboard is the farmer's landing page after login. Its purpose is to surface the health status of every field at a glance, with a clear path to trigger a new analysis or view prior

results.

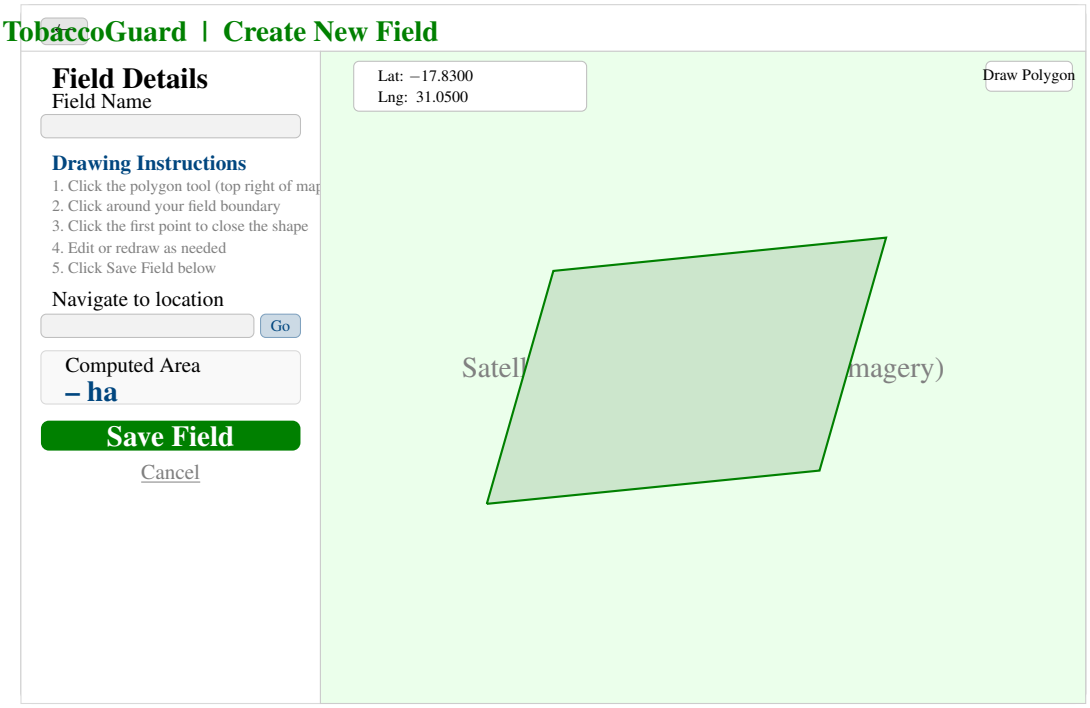
Figure 5.4: Wireframe — Dashboard (Field List View)



5.6.3 Create Field Interface

The Create Field page uses a split layout: a form panel on the left and an interactive satellite map on the right. The map occupies most of the viewport so that drawing a polygon over recognisable aerial imagery is as natural as possible.

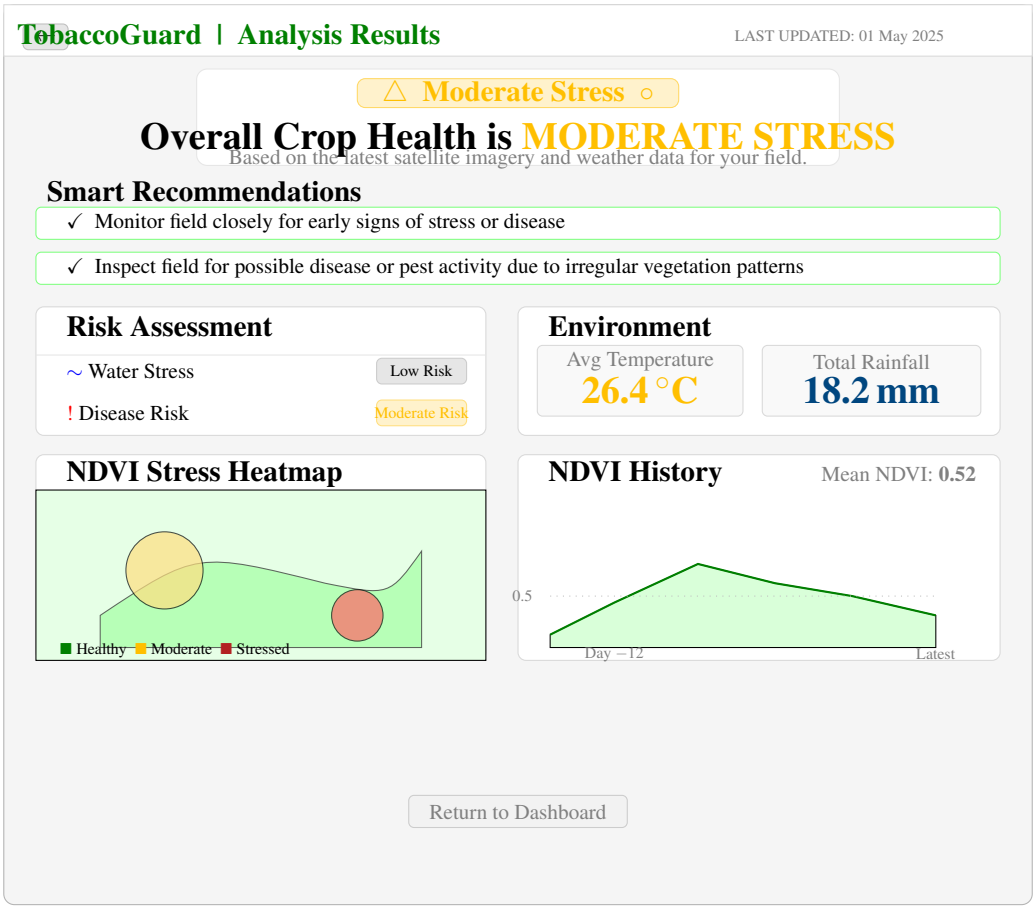
Figure 5.5: Wireframe — Create Field (Interactive Map View)



5.6.4 Analysis Results Interface

The Analysis Results page is the primary output surface of the system. It is designed to present the most important information — the health status and the recommended actions — without requiring the farmer to scroll or navigate further.

Figure 5.6: Wireframe — Analysis Results Page



5.7 Conclusion

The design stage produced a complete specification of the system before a line of implementation code was committed. The four-layer modular architecture separates satellite data retrieval, backend coordination, machine learning intelligence, and browser presentation into components that can be built, tested, and updated independently. The database design supports both the operational and longitudinal requirements of the system through three clearly related entities. The algorithm design provides a step-by-step pathway from field polygon to health classification and spatial heatmap. And the wireframes translate the functional requirements into concrete page layouts that prioritise the farmer's primary information needs. Chapter 6 takes these design decisions and records how each one was realised in working code.

Bibliography

Bégué, A., Arvor, D., Bellon, B., Betbeder, J., de Abelleyra, D., Ferraz, R.P.D., Lebourgeois, V., Lelong, C., Simões, M. and Verón, S.R. (2018) 'Remote sensing and cropping practices: a review', *Remote Sensing*, 10(1), p. 99.

Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5–32.

Chlingaryan, A., Sukkarieh, S. and Whelan, B. (2018) 'Machine learning approaches for crop yield prediction and nitrogen status estimation in precision agriculture: a review', *Computers and Electronics in Agriculture*, 151, pp. 61–69.

Climate Corporation (2023) *Climate FieldView Platform Documentation*. Available at: https://climate.com/fieldview (Accessed: 5 February 2025).

CropIn Technology (2023) *SmartFarm Platform Overview*. Available at: https://www.cropin.com (Accessed: 10 February 2025).

Digital Green (2023) *Digital Green Technology Platform*. Available at: https://www.digitalgreen.org (Accessed: 12 February 2025).

European Space Agency (2023) *Sentinel-2 User Handbook*. 2nd edn. Paris: ESA.

Farmerline (2023) *Farmerline Smart Farming Tools*. Available at: https://farmerline.co (Accessed: 10 February 2025).

Food and Agriculture Organization of the United Nations (2023) *FAOSTAT: Crops and Livestock Products*. Available at: https://www.fao.org/faostat (Accessed: 10 January 2025).

Gebbers, R. and Adamchuk, V.I. (2010) 'Precision agriculture and food security', *Science*, 327(5967), pp. 828–831.

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D. and Moore, R. (2017) 'Google Earth Engine: planetary-scale geospatial analysis for everyone', *Remote Sensing of Environment*, 202, pp. 18–27.

Kamilaris, A. and Prenafeta-Boldú, F.X. (2018) 'Deep learning in agriculture: a survey', *Computers and Electronics in Agriculture*, 147, pp. 70–90.

Liakos, K.G., Busato, P., Moshou, D., Pearson, S. and Bochtis, D. (2018) 'Machine learning in agriculture: a review', *Sensors*, 18(8), p. 2674.

Ministry of Lands, Agriculture, Fisheries, Water and Rural Development (2022) *Zimbabwe Agricultural Development Strategy II (2021–2025)*. Harare: Government of Zimbabwe.

Nyoka, M., Dube, T. and Masocha, M. (2020) 'Assessing the utility of remote sensing data in monitoring tobacco crop health in Zimbabwe', *South African Geographical Journal*, 102(1), pp. 15–32.

Open-Meteo (2023) *Open-Meteo Weather API Documentation*. Available at: https://open-meteo.com/en/docs (Accessed: 20 January 2025).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Perera, C., Liu, C.H. and Jayawardena, S. (2015) 'The emerging Internet of Things marketplace from an industrial perspective: a survey', *IEEE Transactions on Emerging Topics in Computing*, 3(4), pp. 585–598.

Pfleeger, S.L. and Atlee, J.M. (2010) *Software Engineering: Theory and Practice*. 4th edn. Upper Saddle River, NJ: Pearson.

PostgreSQL Global Development Group (2023) *PostgreSQL 16 Documentation*. Available at: https://www.postgresql.org/docs/16 (Accessed: 15 January 2025).

Prisma (2023) *Prisma ORM Documentation*. Available at: https://www.prisma.io/docs (Accessed: 15 January 2025).

Regrow Ag (2023) *Regrow Sustainable Agriculture Platform*. Available at: https://www.regrow.ag (Accessed: 15 February 2025).

Rose, D.C., Sutherland, W.J., Parker, C., Lobley, M., Winter, M., Morris, C., Twining, S., Ffoulkes, C., Amano, T. and Dicks, L.V. (2016) 'Decision support tools for agriculture: towards effective design and delivery', *Agricultural Systems*, 149, pp. 165–174.

Rouse, J.W., Haas, R.H., Schell, J.A. and Deering, D.W. (1974) 'Monitoring vegetation systems in the Great Plains with ERTS', in *Third Earth Resources Technology Satellite-1 Symposium*, vol. 1. Greenbelt, MD: NASA, pp. 309–317.

Rwizi, T.L., Sibanda, M. and Chimonyo, V.G.P. (2021) 'Use of satellite data for agricultural applications in Zimbabwe: opportunities and challenges', *Zimbabwe Journal of Agricultural Sciences*, 3(1), pp. 22–38.

SatAgro (2023) *SatAgro Satellite Monitoring Service*. Available at: https://satagtro.com (Accessed: 12 February 2025).

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson Education.

Taranis (2023) *Taranis Aerial Intelligence Platform*. Available at: https://taranis.ag (Accessed: 5 February 2025).

Tobacco Industry and Marketing Board (2023) *Annual Report 2022/2023*. Harare: TIMB.

Tucker, C.J. (1979) 'Red and photographic infrared linear combinations for monitoring vegetation', *Remote Sensing of Environment*, 8(2), pp. 127–150.

Weiss, M., Jacob, F. and Duveiller, G. (2020) 'Remote sensing for agricultural applications: a meta-review', *Remote Sensing of Environment*, 236, p. 111402.

Wieringa, R.J. (2014) *Design Science Methodology for Information Systems and Software Engineering*. Berlin: Springer.

Wolfert, S., Ge, L., Verdouw, C. and Bogaardt, M.J. (2017) 'Big data in smart farming: a review', *Agricultural Systems*, 153, pp. 69–80.